Mechanics of Flexible Structures & Soft Robots
MAE-263F
Fall 2026

Homework 1

Due Date: 11:59 PM, 10/12/2026

You should create a single GitHub repository for this class and share it with the instructor (`khalidjm@seas.ucla.edu`). All the homeworks, reports, presentations, and proposal should be uploaded to this repository. Do not create a separate repository for each assignment. Within your repository, create a separate folder for each assignment (e.g., `homework_1`, `homework_2`, `homework_3`, `homework_4`, `homework_5`, `proposal`, `midterm_report`, and `final_report`).

Submission Instructions: This homework is graded entirely through the multiple-choice questions on **Gradescope**. In addition, submit the URL to your GitHub repository on BruinLearn. The `homework_1` folder of your repository should include the following items:

1. **Source code:** One main file named exactly `Homework1.py`, along with any additional files (e.g., functions) as necessary. The main file should require no more than a single command to run.
2. **README file:** A `README.md` file with clear instructions on how to run your code and a short description of each file in your repository.

Your code will not be graded, but it must be in your repository. If no code is submitted, the homework receives a score of 0, regardless of your Gradescope answers.

Introduction: A Modular Multi-Mass-Spring-Damper Simulator

In class, we simulated a chain of masses connected by springs and dampers (`massSpringDamper_multi.py`, Chapter 3 of the notes). That code loops over the *masses*. It writes a separate equation of motion for each mass and needs special cases (`if i == 0`, `elif i == N-1`) for the first and last mass. The Jacobian is also filled in one entry at a time. This works for three masses, but it becomes error-prone as the system grows. It also does not carry over to the rods, plates, and shells we will simulate later in the course.

In this homework you will build a cleaner, *modular* version of the same simulator. Instead of looping over masses, you will loop over *springs* and *dampers* (the *elements*). Each element connects two nodes. It adds its own small contribution to the residual vector $\mathbf{f}$ and a small block to the Jacobian $\mathbf{J}$. This "element assembly" structure is the same one we will use for every structure in this course.

Handling the ends of the chain. The end cases are handled once, in a simple way:

- The fixed wall is treated as **node 0** and is included in the position vector. For N springs, the position vector therefore has $N+1$ entries, $x_0, x_1, \ldots, x_N$.

- Spring (and damper) i, for $i = 1, \ldots, N$, **always** connects node $i-1$ and node i. Every element is treated identically, so the first mass is not a special case.
- The last mass is not a special case either; it simply has no spring below it.
- The only boundary condition is that node 0 does not move. After assembly, you solve for the **free** nodes $1, \ldots, N$ only.

What you will do.

1. Implement the functions specified in this handout and check each one against the provided test cases.
2. Use these functions to build an *implicit Euler* simulator for a chain of N springs, dampers, and masses, where N is an input to your code.
3. Implement an *explicit (forward) Euler* time step for the same system.
4. Run your simulators and use the results to answer the multiple-choice questions on **Gradescope**.

The questions are designed so that their answers can be checked against analytical solutions. A correct simulator is the easiest way to answer them with confidence.

Model and Notation

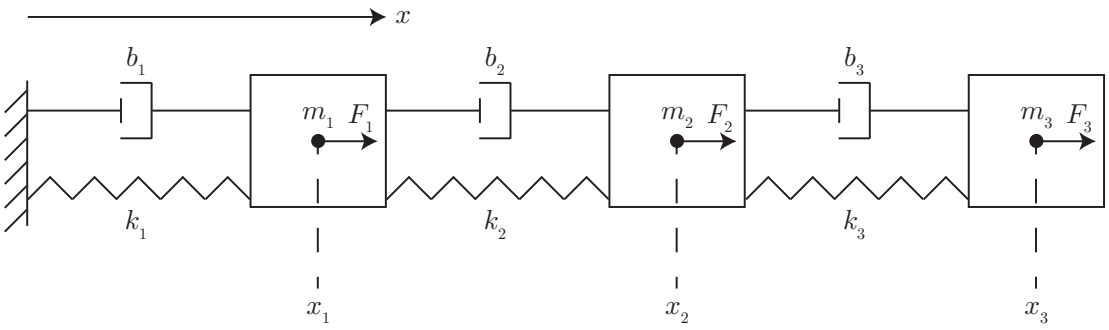


Figure 1: A chain of masses, springs, and dampers in 1D, shown here for $N = 3$. Your code must work for any N.

Fig. 1 shows the system. A chain of N masses moves along the x-axis. Mass m_1 is connected to a fixed wall by spring k_1 and damper b_1. Each mass m_i ($i \geq 2$) is connected to mass m_{i-1} by spring k_i and damper b_i. Each mass is acted on by an external force F_i. **In this homework, the only external force is gravity, which acts along the $+x$ axis**, so $F_i = m_i g$ with $g = 9.8\,\mathrm{m/s^2}$. All quantities are in SI units.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Degrees of freedom. As in Chapter 3 of the notes, x_i is the *displacement* of mass i from its rest position, so all springs are unstretched when every $x_i = 0$. The wall is node 0 and always has $x_0 = 0$. The position and velocity vectors are

$$\mathbf{x} = [x_0, x_1, \ldots, x_N]^T, \qquad \mathbf{u} = \dot{\mathbf{x}} = [u_0, u_1, \ldots, u_N]^T, \tag{1}$$

each with $N + 1$ entries, where $x_0 = u_0 = 0$ at all times. Positive x and positive F_i point to the right.

Notation.

Symbol	Meaning
N	number of springs (= number of dampers = number of masses)
m_i	mass of node i ($i = 1, \ldots, N$)
k_i	stiffness of spring i, connecting node $i-1$ and node i
b_i	viscous damping coefficient of damper i, connecting node $i-1$ and node i
$F_i = m_i g$	external force (gravity) on node i, along $+x$
Δt	time step size
$\mathbf{x}(t_k),\ \mathbf{u}(t_k)$	positions and velocities at time step k

One spring. Spring i connects node $i-1$ and node i. Its stretch is $x_i - x_{i-1}$, and its elastic energy is

$$E_i = \tfrac{1}{2}\, k_i\, (x_i - x_{i-1})^2. \tag{2}$$

The gradient and Hessian of E_i with respect to the two nodal positions (x_{i-1}, x_i) are

$$\nabla E_i = \begin{bmatrix} \partial E_i/\partial x_{i-1} \\ \partial E_i/\partial x_i \end{bmatrix} = k_i(x_i - x_{i-1}) \begin{bmatrix} -1 \\ 1 \end{bmatrix}, \qquad \nabla^2 E_i = k_i \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}. \tag{3}$$

The elastic forces the spring applies to the two nodes are $-\nabla E_i$.

One damper. Damper i also connects node $i-1$ and node i. It resists their relative velocity $u_i - u_{i-1}$, applying the forces $-\mathbf{g}_i$ to the two nodes, where

$$\mathbf{g}_i = b_i(u_i - u_{i-1}) \begin{bmatrix} -1 \\ 1 \end{bmatrix}. \tag{4}$$

Equations of motion. Newton's second law for each free node $j = 1, \ldots, N$ is

$$m_j \ddot{x}_j \;+\; \sum_{i=1}^{N} \frac{\partial E_i}{\partial x_j} \;+\; \sum_{i=1}^{N} (\mathbf{g}_i)_j \;-\; m_j g \;=\; 0, \tag{5}$$

where $(\mathbf{g}_i)_j$ is the entry of $\mathbf{g}_i$ for node j, and is zero if damper i does not touch node j. Only springs and dampers j and $j+1$ touch node j, so most terms in these sums are zero. The equation for node 0 (the wall) is not solved, because $x_0 = 0$ is prescribed.

Check: for $N = 3$ and $j = 1$, Eq. (5) gives $m_1\ddot{x}_1 + (k_1 + k_2)x_1 - k_2 x_2 + (b_1 + b_2)\dot{x}_1 - b_2\dot{x}_2 - m_1 g = 0$, which is Eq. (3.2) of the notes with the external force replaced by gravity.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Algorithm: Implicit Euler with Element Assembly

Discrete equations. We march forward in time with step size Δt. At each step we know $\mathbf{x}^{\text{old}} = \mathbf{x}(t_k)$ and $\mathbf{u}^{\text{old}} = \mathbf{u}(t_k)$, and we solve for $\mathbf{x}^{\text{new}} = \mathbf{x}(t_{k+1})$. As in the notes, the implicit Euler velocity is

$$\mathbf{u}^{\text{new}} = \frac{\mathbf{x}^{\text{new}} - \mathbf{x}^{\text{old}}}{\Delta t}. \tag{6}$$

Discretizing Eq. (5) gives one equation $f_j = 0$ for each node j, where the residual vector $\mathbf{f}$ (with $N+1$ entries) is

$$f_j = \frac{m_j}{\Delta t}\left(\frac{x_j^{\text{new}} - x_j^{\text{old}}}{\Delta t} - u_j^{\text{old}}\right) + \sum_{i=1}^{N} \left.\frac{\partial E_i}{\partial x_j}\right|_{\mathbf{x}^{\text{new}}} + \sum_{i=1}^{N} \left(\mathbf{g}_i\right)_j \Big|_{\mathbf{u}^{\text{new}}} - m_j g. \tag{7}$$

The spring terms are evaluated at the new positions and the damper terms at the new velocities (Eq. 6). The Jacobian $J_{jl} = \partial f_j / \partial x_l^{\text{new}}$ is

$$\mathbf{J} = \frac{1}{\Delta t^2}\,\text{diag}(m_0, m_1, \ldots, m_N) + \sum_{i=1}^{N} \left[\nabla^2 E_i\right] + \sum_{i=1}^{N} \left[\frac{b_i}{\Delta t}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}\right], \tag{8}$$

where $[\cdot]$ means the 2×2 block is added to rows and columns $i-1$ and i of $\mathbf{J}$. The damper block is $b_i/\Delta t$ times the same pattern as the spring Hessian, because $\mathbf{u}^{\text{new}}$ depends on $\mathbf{x}^{\text{new}}$ through the factor $1/\Delta t$. The value of m_0 does not matter (you can set it to 0), because row and column 0 are removed before solving.

Element assembly. The key idea is that the residual and Jacobian are built by looping over the elements. Each spring or damper i touches only nodes $i-1$ and i, so it adds a 2-entry vector to $\mathbf{f}$ and a 2×2 block to $\mathbf{J}$. In Python (0-based indexing, with node 0 as the wall), the loop looks like:

```
for i in range(1, N+1):           # springs/dampers 1, ..., N
    idx = [i-1, i]                # the two nodes this element connects
    # spring
    f[idx] += gradEs(x_new[i-1], x_new[i], k[i])
    J[np.ix_(idx, idx)] += hessEs(k[i])
    # damper
    f[idx] += dampingForce(x_new[i-1], x_new[i],
                           x_old[i-1], x_old[i], b[i], dt)
    J[np.ix_(idx, idx)] += dampingJacobian(b[i], dt)
```

Important: use `J[np.ix_(idx, idx)]`, not `J[idx, idx]`. In NumPy, `J[idx, idx]` selects only the diagonal entries `J[i-1, i-1]` and `J[i, i]`; the off-diagonal entries are silently skipped.

Boundary condition. Node 0 (the wall) is fixed. After assembling the full $(N+1)$-sized $\mathbf{f}$ and $\mathbf{J}$, keep only the *free* degrees of freedom, `free = [1, 2, ..., N]`, and solve

$$\mathbf{J}_{\text{free,free}}\,\Delta\mathbf{x}_{\text{free}} = \mathbf{f}_{\text{free}}, \qquad \mathbf{x}_{\text{free}}^{\text{new}} \leftarrow \mathbf{x}_{\text{free}}^{\text{new}} - \Delta\mathbf{x}_{\text{free}}. \tag{9}$$

x_0^{new} is never updated, so it stays 0.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Pseudocode. Algorithm 1 shows one implicit Euler step, and Algorithm 2 shows the full simulation.

Algorithm 1 `implicitEulerStep`($\mathbf{x}^{\text{old}}, \mathbf{u}^{\text{old}}$, parameters)

$\mathbf{x}^{\text{new}} \leftarrow \mathbf{x}^{\text{old}}$ ▷ initial guess; use `x_old.copy()` in Python
`free` $\leftarrow [1, \ldots, N]$
repeat
  $\mathbf{f}, \mathbf{J} \leftarrow$ `getFJ`($\mathbf{x}^{\text{new}}, \mathbf{x}^{\text{old}}, \mathbf{u}^{\text{old}}$, parameters) ▷ Eqs. (7)–(8), by element assembly
  Solve $\mathbf{J}_{\text{free,free}}\, \Delta\mathbf{x}_{\text{free}} = \mathbf{f}_{\text{free}}$
  $\mathbf{x}^{\text{new}}_{\text{free}} \leftarrow \mathbf{x}^{\text{new}}_{\text{free}} - \Delta\mathbf{x}_{\text{free}}$
  $\mathbf{f} \leftarrow$ residual at the updated $\mathbf{x}^{\text{new}}$; err $\leftarrow \|\mathbf{f}_{\text{free}}\|$
until err $< \epsilon$
$\mathbf{u}^{\text{new}} \leftarrow (\mathbf{x}^{\text{new}} - \mathbf{x}^{\text{old}})/\Delta t$
return $\mathbf{x}^{\text{new}}, \mathbf{u}^{\text{new}}$

Algorithm 2 Main simulation

Set N, m_i, k_i, b_i, g, Δt, total time T, tolerance ϵ
Set initial conditions $\mathbf{x}(0)$, $\mathbf{u}(0)$ (with $x_0 = u_0 = 0$)
for $k = 0, 1, \ldots, T/\Delta t - 1$ **do**
  $\mathbf{x}(t_{k+1}), \mathbf{u}(t_{k+1}) \leftarrow$ `implicitEulerStep`($\mathbf{x}(t_k), \mathbf{u}(t_k)$, parameters)
  Store $\mathbf{x}(t_{k+1})$ and $\mathbf{u}(t_{k+1})$
end for
Plot or post-process the stored positions and velocities

Remark: every spring and damper here is linear, so $\mathbf{f}$ is linear in $\mathbf{x}^{\text{new}}$ and Newton's method converges in a single iteration. We still use the Newton loop because the same code structure is needed later in the course, when the elements become nonlinear.

Required Functions and Test Cases

Conventions. Store the parameters m, k, and b as NumPy arrays of length $N + 1$ so that index i refers to node, spring, or damper i, matching the notation above. Entry 0 is unused; set it to 0. For example, for $N = 3$:

```
m = np.array([0.0, m1, m2, m3])   # m[0] is the wall (unused)
k = np.array([0.0, k1, k2, k3])   # k[i] connects node i-1 and node i
b = np.array([0.0, b1, b2, b3])   # b[i] connects node i-1 and node i
```

Collect all parameters in a dictionary, e.g. `params = {'m': m, 'k': k, 'b': b, 'g': 9.8, 'dt': dt}`. Position and velocity vectors `x` and `u` also have length $N + 1$, with `x[0] = u[0] = 0`.

Functions. Implement the following functions. You may add helper functions, but these six must exist with these inputs and outputs.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Function	Returns
`gradEs(xa, xb, k)`	Gradient of the spring energy, Eq. (3): `k*(xb-xa)*[-1, 1]` (length-2 array)
`hessEs(k)`	Hessian of the spring energy, Eq. (3): `k*[[1,-1],[-1,1]]` (2×2 array)
`dampingForce(xa_new, xb_new, xa_old, xb_old, b, dt)`	Damper force vector $\mathbf{g}_i$, Eq. (4), with velocities computed as $(x^{\text{new}} - x^{\text{old}})/\Delta t$ (length-2 array)
`dampingJacobian(b, dt)`	Derivative of `dampingForce` with respect to `(xa_new, xb_new)`: `(b/dt)*[[1,-1],[-1,1]]` (2×2 array)
`getFJ(x_new, x_old, u_old, params)`	Full residual `f` (length $N+1$) and Jacobian `J` ($(N+1) \times (N+1)$), Eqs. (7)–(8), built by looping over springs and dampers
`implicitEulerStep(x_old, u_old, params)`	`x_new, u_new` after one implicit Euler step (Algorithm 1)

Test cases. Check each function against the cases below before running any simulation. The expected outputs are exact unless shown with six decimals. If a test fails, fix that function before moving on; every later result depends on it.

1. **Spring gradient.** `gradEs(0, 0.1, 10)` → `[-1, 1]`
2. **Spring Hessian.** `hessEs(10)` → `[[10, -10], [-10, 10]]`
3. **Damper force.** `dampingForce(0, 0.01, 0, 0, 2, 0.01)` → `[-2, 2]`. (The second node moves 0.01 m in 0.01 s and the first does not move, so the relative velocity is 1 m/s.)
4. **Damper Jacobian.** `dampingJacobian(2, 0.01)` → `[[200, -200], [-200, 200]]`
5. **Assembly (stiffness only).** $N = 3$, `k = [0, 1, 2, 3]`, `m = b = [0, 0, 0, 0]`, `g = 0`, `dt = 1`, and all positions and velocities zero. The free block `J[1:, 1:]` returned by `getFJ` should be
$$\begin{bmatrix} 3 & -2 & 0 \\ -2 & 5 & -3 \\ 0 & -3 & 3 \end{bmatrix}.$$
This test catches indexing mistakes, including `J[idx, idx]` in place of `J[np.ix_(idx, idx)]`.
6. **Assembly (all terms).** $N = 3$, `m = [0, 1, 2, 3]`, `k = [0, 10, 20, 30]`, `b = [0, 1, 2, 3]`, `g = 9.8`, `dt = 0.1`, `x_old = [0, 0, 0, 0]`, `u_old = [0, 0.1, 0.2, 0.3]`, `x_new = [0, 0.01, 0.03, 0.06]`. Then `getFJ` should give
$$\texttt{f[1:]} = \begin{bmatrix} -10.4 \\ -18.6 \\ -18.6 \end{bmatrix}, \qquad \texttt{J[1:, 1:]} = \begin{bmatrix} 160 & -40 & 0 \\ -40 & 300 & -60 \\ 0 & -60 & 360 \end{bmatrix}.$$

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

7. **One implicit Euler step (free fall).** $N = 1$, `m = [0, 1]`, `k = b = [0, 0]`, `g = 9.8`, `dt = 0.01`, starting from rest at `x_old = u_old = [0, 0]`. Then `implicitEulerStep` should give `x_new = [0, 0.00098]` and `u_new = [0, 0.098]`.

8. **One implicit Euler step (full chain).** Same parameters and `x_old`, `u_old` as Test 6. Then `implicitEulerStep` should give

   `x_new` $= [0,\ 0.096696,\ 0.116786,\ 0.126131]$, `u_new` $= [0,\ 0.966964,\ 1.167857,\ 1.261310]$.

9. **Newton convergence.** In Tests 7 and 8, Newton's method should converge in **exactly one iteration**: after the first update, $\|\mathbf{f}_{\text{free}}\|$ should be at roundoff level ($\sim 10^{-12}$ or smaller). If it takes more iterations, your Jacobian does not match your residual.

A convenient way to run these checks is with `np.allclose`, for example:

```
assert np.allclose(gradEs(0, 0.1, 10), [-1, 1])
assert np.allclose(hessEs(10), [[10, -10], [-10, 10]])
```

Explicit (Forward) Euler

In explicit (forward) Euler, everything on the right-hand side is evaluated at the *old* time step. No system of equations is solved and no Newton iteration is needed.

Update equations. First compute the acceleration of each free node $j = 1, \ldots, N$ from the old positions and velocities:

$$a_j = g \;-\; \frac{1}{m_j}\left(\sum_{i=1}^{N} \frac{\partial E_i}{\partial x_j}\bigg|_{\mathbf{x}^{\text{old}}} \;+\; \sum_{i=1}^{N} (\mathbf{g}_i)_j \,\Big|_{\mathbf{u}^{\text{old}}}\right). \tag{10}$$

This is Eq. (5) solved for $\ddot{x}_j$. Then update the positions and velocities:

$$x_j^{\text{new}} = x_j^{\text{old}} + \Delta t\, u_j^{\text{old}}, \tag{11}$$

$$u_j^{\text{new}} = u_j^{\text{old}} + \Delta t\, a_j. \tag{12}$$

The wall stays fixed: $x_0^{\text{new}} = u_0^{\text{new}} = 0$.

The sums in Eq. (10) are assembled with the same loop over springs and dampers as in the implicit code. You can reuse `gradEs` directly with the old positions. For the damper, use Eq. (4) with the old velocities, $\mathbf{g}_i = b_i\,(u_i^{\text{old}} - u_{i-1}^{\text{old}})\,[-1,\ 1]^T$.

Note: this is not Eq. (2.3) of the notes. In Eq. (2.3) of the notes, the new velocity is defined as $(x^{\text{new}} - x^{\text{old}})/\Delta t$, which makes the position update use the *new* velocity. That is a different method, with different stability behavior. In this homework, use Eqs. (11)–(12) exactly as written: both updates use only old values.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Function and test cases. Implement

`explicitEulerStep(x_old, u_old, params)` → `x_new, u_new`

with the same inputs and outputs as `implicitEulerStep`, so that switching your simulator from one method to the other only requires changing one line.

10. **One explicit step (single spring).** $N = 1$, `m = [0, 1]`, `k = [0, 1]`, `b = [0, 0]`, `g = 0`, `dt = 0.1`, `x_old = [0, 1]`, `u_old = [0, 0]`. Then `explicitEulerStep` should give `x_new = [0, 1]` and `u_new = [0, -0.1]`.
    (The position does not change in the first step, because it is updated with the old velocity, which is zero.)

11. **One explicit step (full chain).** Same parameters as Test 6 (`m = [0, 1, 2, 3]`, `k = [0, 10, 20, 30]`, `b = [0, 1, 2, 3]`, `g = 9.8`, `dt = 0.1`), with `x_old = [0, 0.01, 0.03, 0.06]` and `u_old = [0, 0.1, 0.2, 0.3]`. Then `explicitEulerStep` should give

    `x_new` $= [0,\ 0.02,\ 0.05,\ 0.09]$, `u_new` $= [0,\ 1.12,\ 1.21,\ 1.24]$.

Questions (answer on Gradescope)

Answer the following multiple-choice questions on Gradescope using your simulators. Unless a question says otherwise:

- gravity $g = 9.8\,\mathrm{m/s^2}$ acts along $+x$;
- all nodes start **at rest** at $x_i = 0$ (all springs unstretched);
- use **implicit Euler** with $\Delta t = 0.01\,\mathrm{s}$;
- parameters are listed for $i = 1, \ldots, N$ (e.g., $m = [1, 2, 3]$ means $m_1 = 1$, $m_2 = 2$, $m_3 = 3$);
- choose the option **closest** to your result.

Every question has an analytical answer, so you can use it to check your code.

Running time: do not plot inside the time loop (as `massSpringDamper_multi.py` does with `plt.pause`); store the results and plot after the loop. Each run should take at most a few seconds.

Part A: Free fall

1. $N = 1$, $m = 1$, $k = 0$, $b = 0$, $\Delta t = 10^{-3}\,\mathrm{s}$. What is x_1 at $t = 1\,\mathrm{s}$?
    (a) 0
    (b) 2.45 m
    (c) 4.9 m
    (d) 9.8 m

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

2. $N = 3$, $m = [1, 2, 3]$, $k = [0, 0, 0]$, $b = [0, 0, 0]$, $\Delta t = 10^{-3}\,\mathrm{s}$. What is x_3 at $t = 2\,\mathrm{s}$?

   (a) $9.8\,\mathrm{m}$
   (b) $19.6\,\mathrm{m}$
   (c) $39.2\,\mathrm{m}$
   (d) $58.8\,\mathrm{m}$

Part B: Terminal velocity

3. $N = 1$, $m = 1$, $k = 0$, $b = 2$. What is the velocity u_1 at $t = 20\,\mathrm{s}$?

   (a) $2.45\,\mathrm{m/s}$
   (b) $4.9\,\mathrm{m/s}$
   (c) $9.8\,\mathrm{m/s}$
   (d) $19.6\,\mathrm{m/s}$

4. $N = 2$, $m = [1, 1]$, $k = [0, 0]$, $b = [2, 2]$. What is the velocity u_2 at $t = 20\,\mathrm{s}$?

   (a) $2.45\,\mathrm{m/s}$
   (b) $4.9\,\mathrm{m/s}$
   (c) $9.8\,\mathrm{m/s}$
   (d) $14.7\,\mathrm{m/s}$

Part C: Static equilibrium

5. $N = 1$, $m = 1$, $k = 10$, $b = 1$. What is x_1 at $t = 50\,\mathrm{s}$?

   (a) $0.98\,\mathrm{m}$
   (b) $1.96\,\mathrm{m}$
   (c) $4.9\,\mathrm{m}$
   (d) $9.8\,\mathrm{m}$

6. $N = 5$, $m = [1, 1, 1, 1, 1]$, $k = [10, 10, 10, 10, 10]$, $b = [5, 5, 5, 5, 5]$. What is the position of the last mass, x_5, at $t = 100\,\mathrm{s}$?

   (a) $2.94\,\mathrm{m}$
   (b) $4.9\,\mathrm{m}$
   (c) $14.7\,\mathrm{m}$
   (d) $29.4\,\mathrm{m}$

7. $N = 3$, $m = [1, 2, 3]$, $k = [10, 20, 30]$, $b = [5, 10, 15]$. What is x_3 at $t = 100\,\mathrm{s}$?

   (a) $5.88\,\mathrm{m}$
   (b) $9.31\,\mathrm{m}$
   (c) $11.76\,\mathrm{m}$
   (d) $17.64\,\mathrm{m}$

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Part D: Artificial damping in implicit Euler

For Questions 8 and 9, use $N = 1$, $m = 1$, $k = 1$, and **no damping** ($b = 0$). Starting from rest at $x_1 = 0$, the exact solution is $x_1(t) = 9.8\,(1 - \cos t)$: the mass oscillates between 0 and 19.6 m forever.

8. Simulate with implicit Euler and $\Delta t = 0.1\,\text{s}$. What is x_1 at $t = 100\,\text{s}$?

   (a) 0
   (b) 9.8 m
   (c) 19.6 m
   (d) Larger than 1000 m (the simulation diverges)

9. Repeat Question 8 with $\Delta t = 0.01\,\text{s}$. Compared with $\Delta t = 0.1\,\text{s}$, the oscillation near $t = 100\,\text{s}$ is:

   (a) Larger: the artificial damping is weaker with a smaller Δt.
   (b) Smaller: the artificial damping is stronger with a smaller Δt.
   (c) The same: the artificial damping does not depend on Δt.
   (d) Growing without bound.

Part E: Explicit Euler

For Questions 10 and 11, use the same system as Part D ($N = 1$, $m = 1$, $k = 1$, $b = 0$, starting from rest at $x_1 = 0$), but simulate with **explicit (forward) Euler**, Eqs. (11)–(12), with $\Delta t = 0.1\,\text{s}$.

10. Which statement best describes $x_1(t)$ for $0 \le t \le 100\,\text{s}$?

    (a) It oscillates with a growing amplitude (it grows without bound).
    (b) It decays toward 9.8 m, as with implicit Euler.
    (c) It oscillates steadily between 0 and 19.6 m, matching the exact solution.
    (d) It stays at 0, because the position is always updated with the old velocity.

11. What is the maximum value of x_1 over $0 \le t \le 10\,\text{s}$?

    (a) 9.8 m
    (b) 18.2 m
    (c) 19.6 m
    (d) 25.5 m

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

Appendix: Analytical Solutions

The derivations below are symbolic. Use them to check your simulation results; substituting the parameters of each question gives the expected answer.

A. Free fall ($k_i = 0$, $b_i = 0$)

With no springs or dampers, the nodes are decoupled, and Eq. (5) for each node reduces to $m_j \ddot{x}_j = m_j g$, i.e. $\ddot{x}_j = g$. Starting from rest at $x_j = 0$,

$$x_j(t) = \tfrac{1}{2}\, g\, t^2, \tag{13}$$

for every node, independent of m_j and of N.

Implicit Euler error. With $u_{n+1} = u_n + g\,\Delta t$ and $x_{n+1} = x_n + \Delta t\, u_{n+1}$, after n steps ($t_n = n\Delta t$),

$$x_n = g\,\Delta t^2 \sum_{l=1}^{n} l = \tfrac{1}{2}\, g\, t_n^2 + \tfrac{1}{2}\, g\, t_n\, \Delta t.$$

The extra term is the $\mathcal{O}(\Delta t)$ error of the method; it vanishes as $\Delta t \to 0$.

B. Terminal velocity ($k_i = 0$, $b_i > 0$)

One mass. The equation of motion is $m\dot{u} = mg - bu$. Starting from rest,

$$u(t) = \frac{mg}{b}\left(1 - e^{-bt/m}\right) \;\longrightarrow\; u_\infty = \frac{mg}{b} \quad \text{for } t \gg m/b. \tag{14}$$

A chain of dampers. At terminal velocity every acceleration is zero, so each damper carries the full weight of all masses below it. Damper i connects nodes $i-1$ and i, so

$$b_i\,(u_i - u_{i-1}) = g \sum_{l=i}^{N} m_l \qquad \Longrightarrow \qquad u_j = \sum_{i=1}^{j} \frac{g}{b_i} \sum_{l=i}^{N} m_l, \tag{15}$$

with $u_0 = 0$. For $N = 2$: $u_1 = (m_1 + m_2)\, g/b_1$ and $u_2 = u_1 + m_2\, g/b_2$.

C. Static equilibrium ($k_i > 0$)

When the motion has died out (damping only controls how fast this happens), all velocities and accelerations are zero. By the same argument as in B, spring i carries the weight of all masses below it:

$$k_i\,(x_i - x_{i-1}) = g \sum_{l=i}^{N} m_l \qquad \Longrightarrow \qquad x_j = \sum_{i=1}^{j} \frac{g}{k_i} \sum_{l=i}^{N} m_l, \tag{16}$$

with $x_0 = 0$. For a single mass, $x_1 = mg/k$. For a uniform chain ($m_i = m$, $k_i = k$), spring i carries $(N - i + 1)\, mg$, so

$$x_N = \frac{mg}{k} \sum_{i=1}^{N} (N - i + 1) = \frac{mg}{k}\, \frac{N(N+1)}{2}. \tag{17}$$

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor

D. Undamped oscillator: exact solution and implicit Euler

Take $N = 1$, $b = 0$, and let $\omega = \sqrt{k/m}$. Measure the position from the static equilibrium, $y = x - mg/k$, so that the equation of motion becomes $\ddot{y} = -\omega^2 y$.

Exact solution. Starting from rest at $x = 0$ (i.e. $y(0) = -mg/k$),

$$x(t) = \frac{mg}{k}\big(1 - \cos\omega t\big). \tag{18}$$

The mass oscillates between 0 and $2mg/k$ forever. A convenient way to track the amplitude is the complex variable $w = \omega y + i\,u$: the oscillation amplitude is $|w|/\omega$, and the total energy is $\frac{1}{2}ky^2 + \frac{1}{2}mu^2 = \frac{1}{2}m|w|^2$. For the exact solution, $|w|$ is constant.

Implicit Euler. The discrete equations are

$$y_{n+1} = y_n + \Delta t\, u_{n+1}, \qquad u_{n+1} = u_n - \omega^2 \Delta t\, y_{n+1}.$$

Multiplying the first by ω and the second by i, adding, and collecting the $(n+1)$ terms on the left gives

$$(1 + i\,\omega\Delta t)\, w_{n+1} = w_n \qquad \Longrightarrow \qquad |w_{n+1}| = \frac{|w_n|}{\sqrt{1 + \omega^2\Delta t^2}}. \tag{19}$$

The amplitude shrinks by the same factor every step. After time $t = n\Delta t$,

$$\frac{A(t)}{A(0)} = \big(1 + \omega^2\Delta t^2\big)^{-t/(2\Delta t)} \;\approx\; e^{-\omega^2 \Delta t\, t/2}. \tag{20}$$

So implicit Euler always damps the oscillation, even though $b = 0$, and x settles to the static value mg/k. The artificial decay rate $\omega^2\Delta t/2$ is proportional to Δt: a smaller time step gives weaker artificial damping, and the exact (undamped) solution is recovered as $\Delta t \to 0$.

E. Undamped oscillator with explicit (forward) Euler

With Eqs. (11)–(12), both updates use old values:

$$y_{n+1} = y_n + \Delta t\, u_n, \qquad u_{n+1} = u_n - \omega^2 \Delta t\, y_n.$$

Combining them in the same way gives

$$w_{n+1} = (1 - i\,\omega\Delta t)\, w_n \qquad \Longrightarrow \qquad |w_{n+1}| = \sqrt{1 + \omega^2\Delta t^2}\;|w_n|. \tag{21}$$

The amplitude now *grows* by the same factor every step:

$$\frac{A(t)}{A(0)} = \big(1 + \omega^2\Delta t^2\big)^{t/(2\Delta t)} \;\approx\; e^{\omega^2 \Delta t\, t/2}. \tag{22}$$

Since $\sqrt{1 + \omega^2\Delta t^2} > 1$ for every $\Delta t > 0$, the undamped oscillator is unstable with forward Euler for *any* time step; a smaller Δt only slows the growth. Starting from rest at $x = 0$, $A(0) = mg/k$, so the oscillation about mg/k has amplitude $A(t)$ and

$$x(t) \approx \frac{mg}{k} - A(t)\cos\omega t, \tag{23}$$

whose peaks occur near $\omega t = \pi, 3\pi, 5\pi, \ldots$, with values $\frac{mg}{k} + A(t)$.

Comparison. Implicit Euler multiplies the amplitude by $(1+\omega^2\Delta t^2)^{-1/2}$ per step, and forward Euler multiplies it by $(1 + \omega^2\Delta t^2)^{+1/2}$. One removes energy, and the other adds it.

Copyright notice: Do not copy, distribute, republish, upload, post or transmit anything from the files hosted on the class website without written permission of the instructor